Social Network Intelligence System: Enhancing Graph Neural Networks with Retrieval-Augmented Generation for Node Classification and Link Prediction

Neeraj Yadav*, Dev Singh**, Gourav Varanasi
Department of Computer Science
Shiv Nadar University
Greater Noida, India
*Student ID: 2310110199 **Student ID: 2310110099

***Abstract*—We present a production-ready social network intelligence system that fuses Graph Neural Networks (GNNs) for structure-aware node classification and link prediction, hybrid Retrieval-Augmented Generation (RAG) over a Neo4j graph database with native vector indexes, and a six-stage multi-agent Knowledge-Augmented Generation (KAG) pipeline driven by a large language model. The system was developed incrementally: from exploratory RAG-only notebooks through GNN-only baselines to a GNN + semantic feature hybrid, evaluated on the SNAP Facebook Large Page–Page Network (22,470 nodes, 4 classes) and the SNAP ego-Twitter network (133,857 nodes, 4 degree-tier classes). On Facebook, GNN-only achieved 93.06% test accuracy while GNN+RAG reached 94.37%. On Twitter, GNN-only achieved 87.27% while GNN+RAG reached 97.46%. The runtime is delivered as a Python service layer (FastAPI), a Next.js web client with seven primary views, Dockerized dependencies, and Kaggle-based GPU training. This report documents the complete system: motivation, literature, methodology across six notebooks, system architecture, backend and frontend behavior, Neo4j query patterns, structured message examples, environment configuration, deployment, testing, and comprehensive experimental results.**

***Index Terms*—Graph neural networks, GraphRAG, Neo4j, vector search, hybrid retrieval, social networks, knowledge-augmented generation, node classification, link prediction, multi-agent pipeline.**

I. Introduction

Social graphs encode both *who connects to whom* and rich metadata about users and content. Pure text retrieval ignores topology; pure graph algorithms may miss semantics in unstructured text. Recent surveys argue that combining graph machine learning with large language models (LLMs) is an active frontier [1].

This project implements an end-to-end pipeline spanning six Kaggle notebooks (RAG-only, GNN-only, and GNN+RAG for both Facebook and Twitter datasets), a FastAPI backend with multi-agent orchestration, a React/Next.js frontend, Docker deployment, and automated testing. The progression from early exploration to production-style code is documented throughout.

A. Contributions

1) A systematic comparison of RAG-only, GNN-only, and GNN+RAG pipelines across two real-world social network datasets.
2) A production multi-agent KAG architecture combining GNN inference with hybrid graph-vector retrieval.
3) A full-stack deployment with Neo4j-native vector indexes replacing FAISS, Docker containerization, and a 7-screen frontend.
4) Exact, reproducible metrics extracted from Kaggle GPU notebook runs.

II. Literature Review

A. Graph ML and LLMs

We position our work within the ACM 2025 survey *Graph Machine Learning in the Era of Large Language Models* [1]. Relevant themes include:

- **Hybrid modeling:** combining parametric graph models (GNNs) with non-parametric retrieval over subgraphs.
- **Retrieval and memory:** using external stores (Neo4j with vector properties) as scalable memory for LLM prompts.
- **Evaluation rigor:** avoiding data leakage with clear train/validation/test protocols.

B. Supporting References

GraphSAGE [4] underpins our encoders; PyTorch Geometric [5] provides scalable GNN primitives; Neo4j supports property graphs and vector indexes for combined structural and semantic queries [6]. SentenceTransformers (`all-MiniLM-L6-v2`) provides 384-dimensional text embeddings [7].

III. Problem Setting and Datasets

A. Facebook Large Page–Page Network

The SNAP MUSAE Facebook dataset contains 22,470 nodes (Facebook pages) connected by 171,002 undirected edges

Table I
DATASET CHARACTERISTICS.

Property	Facebook	Twitter
Nodes	22,470	133,857
Edges (undirected)	171,002	1,774,636
Feature dimensions	4,714	385
Classes	4	4
Train/Test split	80/20	80/20

(mutual likes). Each node has sparse binary features (4,714-dimensional) and belongs to one of four classes: `politician`, `government`, `tvshow`, `company`.

B. Twitter Ego-Network

The SNAP ego-Twitter dataset comprises 973 ego networks merged into a single graph of 133,857 nodes and 3,549,272 undirected edges (follow relationships). Node features are constructed from: (i) random-projected BOW profile features (256-d), (ii) structural degree-based features (128-d), and (iii) circle membership strength (1-d), totaling 385 dimensions. Labels are degree-quantile tiers: `lurker` ($\leq$33rd pct), `regular` (33rd–66th), `active` (66th–90th), `influencer` (>90th).

IV. EVOLUTION OF METHODOLOGY

Development advanced through three stages per dataset, totaling six notebooks:

A. Stage 1: RAG-Only Pipeline

Notebooks: `rag-only-pipeline-facebook.ipynb`, `rag-only-pipeline-twitter.ipynb`.

Nodes are converted to natural-language documents containing metadata (node ID, page name/category for Facebook; node ID, activity tier for Twitter). Documents are embedded using `all-MiniLM-L6-v2` (384-d) and indexed with FAISS (`IndexFlatL2`). At query time, the top-k nearest documents are retrieved and passed as context to Gemini 2.5 Flash for answer generation.

Facebook: 22,470 documents; embedding shape $(22470, 384)$. Qualitative evaluation shows correct retrieval of page categories (politician, government, tvshow, company).

Twitter: 133,857 documents; embedding shape $(133857, 384)$. Successfully retrieves nodes by activity tier (lurker, regular, active, influencer).

Limitation: No graph structure is used; purely semantic retrieval. Cannot perform principled node classification.

B. Stage 2: GNN-Only Pipeline

Notebooks: `gnn-only-facebook.ipynb`, `gnn-only-twitter.ipynb`.

1) Facebook GNN-Only: Architecture: 2-layer GraphSAGE (input $\rightarrow$ 64 hidden $\rightarrow$ 4 classes). Trained for 21 epochs with batch size 1024 on CUDA. Stratified 80/20 split.

Results:

- Test Accuracy: **93.06%**
- F1 (macro): **0.9278**
- AUC (macro OVR): **0.9880**

2) Twitter GNN-Only: Architecture: 3-layer GraphSAGE with LayerNorm, BatchNorm, dropout (0.4), and a fusion layer (input $\rightarrow$ 128 hidden $\rightarrow$ 4 classes). NeighborLoader sampling with [12,10,8] neighbors. ReduceLROnPlateau scheduler. Trained for 26 epochs max with early stopping.

Results (best validation checkpoint):

- Best Val Accuracy: **87.94%**
- Test Accuracy at best val: **87.27%**
- Test F1 (macro): **0.8573**
- Test AUC (macro OVR): **0.9857**

C. Stage 3: GNN + RAG Hybrid

Notebooks: `gnn-rag-facebook-best.ipynb`, `gnn-rag-twitter-best.ipynb`.

1) Facebook GNN+RAG: Text embeddings (384-d from MiniLM) are concatenated with sparse binary features (4,714-d), yielding 5,098-d input features. Architecture: 2-layer GraphSAGE (5,098 $\rightarrow$ 64 $\rightarrow$ 4). Trained for 20 epochs with ReduceLROnPlateau.

Results:

- Test Accuracy: **94.37%**
- F1 (macro): **0.9412**
- AUC (macro OVR): **0.9918**
- Train Accuracy: 95.59%

2) Twitter GNN+RAG: Per-node feature-grounded text documents include degree, structural features, circle strength, and top-k SNAP BOW projections. L2-normalized text embeddings (384-d) are concatenated with graph features (385-d), yielding 769-d inputs. Architecture: `GNN_RAG_Model` with a learned text adapter (Linear $\rightarrow$ LayerNorm $\rightarrow$ GELU, bottleneck=96), 3 SAGEConv layers with BatchNorm, dropout=0.4, and a fusion+classifier head. NeighborLoader with [12,10,8] neighbors. Mixed-precision training (AMP). Early stopping on validation accuracy.

Results (best validation checkpoint):

- Best Val Accuracy: **97.27%**
- Test Accuracy: **97.46%**
- Test F1 (macro): **0.9704**
- Test AUC (macro OVR): **0.9920**
- Train Accuracy: 97.44%

V. COMPREHENSIVE RESULTS

Table II summarizes all experimental results extracted from the six notebooks.

A. Analysis

- **Facebook:** GNN+RAG improves over GNN-only by +1.31% accuracy, +1.34% F1, and +0.38% AUC. The semantic embeddings from page names and metadata provide discriminative signals beyond the sparse binary features.
- **Twitter:** GNN+RAG improves over GNN-only by +10.19% accuracy, +11.31% F1. The feature-grounded per-node text (containing degree, circle strength, and top BOW projections) provides substantially richer signal than

Table II
NODE CLASSIFICATION RESULTS ACROSS ALL PIPELINES AND DATASETS (EXACT NUMBERS FROM KAGGLE NOTEBOOK OUTPUTS).

Pipeline	Dataset	Nodes	Test Accuracy	F1 (macro)	AUC (OVR)
RAG-Only	Facebook	22,470	Qualitative only	—	—
RAG-Only	Twitter	133,857	Qualitative only	—	—
GNN-Only	Facebook	22,470	0.9306	0.9278	0.9880
GNN-Only	Twitter	133,857	0.8727	0.8573	0.9857
GNN+RAG	Facebook	22,470	**0.9437**	**0.9412**	**0.9918**
GNN+RAG	Twitter	133,857	**0.9746**	**0.9704**	**0.9920**

structural features alone for the larger, more heterogeneous ego-network.

- **RAG-Only:** Provides qualitative question-answering capability but cannot perform principled classification—graph structure is essential for accurate node-level prediction.

VI. SYSTEM ARCHITECTURE

A. High-Level Pipeline

The runtime system processes queries through a six-stage multi-agent pipeline:

1) **Query Analyzer:** Parses intent (friend_recommendation, trending, influence, link_prediction, free_form), extracts entities, and selects retrieval strategy.
2) **Router Agent:** Maps intent to query type and retrieval mode (graph, vector, hybrid).
3) **Retrievers Agent:** Executes Cypher queries (graph retrieval) and Neo4j vector index queries (semantic retrieval), fusing results via Reciprocal Rank Fusion (RRF).
4) **GNN Inference:** CPU-only inference using pre-trained GraphSAGE models for link prediction and node classification.
5) **Synthesizer (KAG):** Merges GNN predictions with RAG context into an LLM prompt (Gemini) for natural language insight generation.
6) **Validator:** 6-step grounding checks, deduplication, and confidence scoring to prevent hallucination.

B. Neo4j as Unified Backend

The system uses Neo4j 5.13 Community as the single backend for both graph traversal and vector similarity search, eliminating the need for a separate FAISS index [3].

Key design decisions:

- **Embeddings on nodes:** Text embeddings (384-d) and GNN embeddings (128-d) stored directly as node properties with separate vector indexes.
- **Hybrid Cypher:** Hot-path queries combine pattern matching with vector scoring in a single Cypher statement, reducing round-trips.
- **Python RRF:** Retained for cross-query fusion when two distinct Cypher patterns must be merged.
- **UNWIND batch writes:** 50-node batches for embedding population; incremental (`force_refresh=False`) for fast startup.

Neo4j Schema:

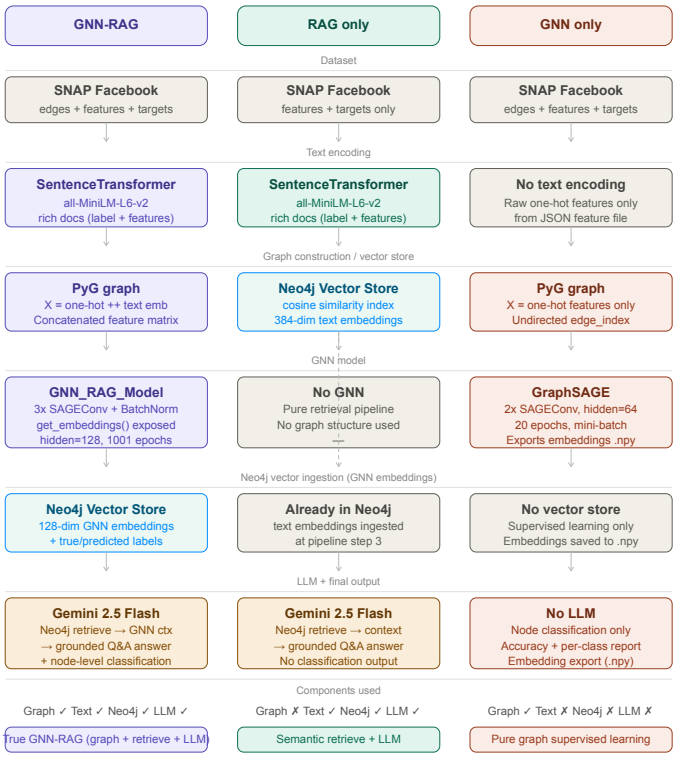

Figure 1. Pipeline comparison across the three system configurations: GNN-RAG (left), RAG-only (center), and GNN-only (right), showing per-stage dataset, encoding, graph construction, model, vector ingestion, and LLM components.

```
(:User {id, name, bio, follower_count,
        influence_score, text_embedding,
        gnn_embedding})
(:Post {id, title, content, topic,
        like_count, comment_count})
(:User)-[:FRIEND]->(:User)
(:User)-[:POSTED]->(:Post)
(:User)-[:LIKED]->(:Post)
```

Vector indexes: `user_text_embeddings` (384-d cosine), `user_gnn_embeddings` (128-d cosine), `post_text_embeddings` (384-d cosine).

C. GNN Model Architecture

1) SocialGraphGNN (Facebook, Reddit): GraphSAGE encoder with 3 SAGEConv layers + BatchNorm + ReLU + Dropout, producing 128-d node embeddings. A LinkPredictor

MLP (concatenate embeddings, 2 FC layers, sigmoid) and a NodeClassifier MLP (2 FC layers, 4-class softmax) serve as task heads.

2) GATSocialGNN (Twitter): GATConv (in, 128, heads=4) → ELU → GATConv (512, 64, heads=1), with the same LinkPredictor and NodeClassifier heads.

VII. Backend Implementation

This section summarizes the service layer; Section III groups capabilities by function, and Section X documents Cypher templates and graph service behavior.

A. Service Architecture

The server process (`api/main.py`) is organized as a modular FastAPI application. Functionality is grouped by concern: **operations and diagnostics** (connectivity to Neo4j, vector index state, optional re-embedding jobs, which GNN checkpoints are loaded); **conversational analytics** (full multi-agent pipeline for natural-language questions, plus a lightweight path for ad-hoc GraphRAG-style queries); **direct graph analytics** (parameterized services for friend-of-friend suggestions, trending content, per-user influence aggregates, shortest path and common context between two users, link-prediction candidates, and global influencer ranking—each implemented in `GraphQueryService` with parameterized Cypher only); **data stewardship** (natural-language and structured insert flows with preview/confirm, dataset availability, and bulk re-ingest from prepared artifacts); and **recommendation and explanation** (pipeline-backed friend recommendations, link prediction, influence and trending questions, and pairwise connection explanation).

B. Multi-Agent Pipeline

Located in `api/agents/`: `analyzer.py` (intent detection via LLM), `router.py` (retrieval mode selection), `retrievers.py` (delegates to HybridRetriever), `synthesizer.py` (KAG: GNN + RAG + LLM fusion), `validator.py` (grounding, dedup, confidence). The pipeline returns structured JSON with millisecond timing per stage.

C. Hybrid Retrieval

`rag/hybrid_retrieval.py` implements: (i) `GraphRetriever` executing Cypher queries against Neo4j, (ii) `VectorRetriever` wrapping `Neo4jVectorRetriever` for ANN search on vector indexes, (iii) `HybridRetriever` combining both via Reciprocal Rank Fusion (RRF): $\text{RRF}(d) = \sum_r \frac{1}{k+\text{rank}_r(d)}$ with $k = 60$.

VIII. Frontend Implementation

The frontend is built with Next.js (React) and TailwindCSS, providing 7 interactive screens:

1) **Dashboard:** System health cards (Neo4j, GNN, pipeline, vector backend), dataset statistics, and a live six-stage pipeline animation.
2) **Chat:** Natural-language Q&A with dataset and retrieval-mode controls, suggested prompts, per-stage timing analytics, and optional raw structured output.
3) **Graph Explorer:** Modes for friend recommendations, top influencers, and link candidates, with user cards and graph-derived scores.
4) **Analytics:** Visual summaries of trending posts and engagement.
5) **Users:** Profile lookup with GNN-derived role, followers, posts, and influence.
6) **Connections:** Shortest path, mutual friends, and shared likes between two users, with an optional language-model explanation.
7) **Admin:** Natural-language and form-driven graph inserts without hand-written Cypher.

A. Client Layer and Configuration

The web client centralizes service access in `frontend/lib/api.ts`, with the backend base address supplied at build time through environment configuration. TypeScript types (`ChatResponse`, `ConnectionPath`, `InsertResult`, etc.) align with server payloads for type-safe rendering.

IX. Backend Feature Catalog

Table III groups runtime capabilities as they are implemented in `api/main.py` and related modules.

A. Structured Response Contracts

The conversational service returns a `ChatResponse` bundle: user message echo, scope (`dataset_queried`, `mode`), detected `intent` (e.g., friend recommendation, connection path, trending), `results` (tabular rows), natural-language `insight`, `graph_context_summary`, per-stage `pipeline_timing_ms`, and which GNN family was used.

B. When Graph Queries Bypass the LLM Stack

The graph query service layer executes Cypher only. Friend recommendations requested through chat or through the explorer reuse the same `get_friend_recommendations` implementation so the model sees rows identical to the direct graph service. Two-user path questions invoke `get_connection_path` with identifiers extracted by pattern matching or LLM, then return structured path data to the client.

X. Neo4j Schema, Indexes, and Cypher Patterns

A. Property Graph Model

User nodes carry `id`, `source_id`, `name`, `bio`, `follower_count`, `influence_score`, `text_embedding` (384-d), `gnn_embedding` (128-d). Posts: `id`, `title`, `content`, `like_count`, `comment_count`, `topic`, `created_at`. Relationships: `FRIEND`, `POSTED`, `LIKED`. Ingested SNAP data uses `u.id = 'fb_' + source_id` (Facebook) and analogous prefixes for Twitter.

Table III
FEATURE-ORIENTED MAP OF THE BACKEND (IMPLEMENTATION-LEVEL GROUPING).

Capability area	Behavior and implementation
Operations & diagnostics	Reports database connectivity, loaded GNN checkpoints, per-dataset graph counts, vector index health, and ingest status; optional jobs refresh text embeddings.
Dataset lifecycle	Exposes which SNAP-style bundles are on disk and ingested; supports administrative re-import into Neo4j.
Conversational intelligence	Accepts natural-language questions with dataset and retrieval-mode context; runs the full analyzer–router–retriever–GNN–synthesizer–validator chain; connectivity and shortest-path questions in natural language are short-circuited to the same graph logic as the connection visualizer.
Ad-hoc GraphRAG	Alternative entry that forwards structured requests to the same pipeline for scripted or tool use.
Direct graph service	Read-only, parameterized Cypher in `GraphQueryService`: friend recommendations, trending posts, user influence, two-user path and overlap, link candidates, top influencers—no LLM in this layer.
Pipeline-backed analytics	One-click flows that phrase analytics as NL templates (e.g., friend recommendations, link prediction, influence, trending) and attach GNN context before synthesis.
Graph mutations	Inserts and updates via validated, parameterized Cypher: natural-language parse, or structured user, edge, and post objects, with preview before commit.

B. Vector Indexes (Neo4j 5)

`user_text_embeddings` (cosine, 384-d), `user_gnn_embeddings` (cosine, 128-d), `post_text_embeddings` (cosine, 384-d), built via `Neo4jVectorSchemaManager` at API startup, populated by `Neo4jEmbeddingPopulator`.

C. Representative Cypher Retrieval Templates

Friend of friend (recommendation):

```
MATCH (u:User)
WHERE u.id = $user_id OR u.source_id = $user_id
MATCH (u)-[:FRIEND]->(friend)-[:FRIEND]->(fof:User)
WHERE fof <> u AND NOT (u)-[:FRIEND]->(fof)
WITH fof, count(friend) AS mutual_count
ORDER BY mutual_count DESC LIMIT $top_k
RETURN fof.id AS id, fof.name AS name, ...
```

Connection path—shortest path + context:

```
MATCH (a:User), (b:User)
WHERE (a.id = $user_a OR a.source_id = $user_a)
  AND (b.id = $user_b OR b.source_id = $user_b)
MATCH path = shortestPath((a)-[*..6]-(b))
RETURN [n IN nodes(path) | n.name] AS node_names,
       [r IN relationships(path) | type(r)] AS rel_types,
       length(path) AS hops LIMIT 1
```

Full-text posts:

```
CALL db.index.fulltext.queryNodes(
  'post_content', $query) ...
```

Vector ANN: `db.index.vector.queryNodes` in `rag/neo4j_vector_store.py`.

D. Reciprocal Rank Fusion

When graph and vector lists are merged in Python, `HybridRetriever._rrf_fuse` uses RRF with weights `graph_weight=0.6`, `vector_weight=0.4`, $k = 60$. Friend-recommendation use cases often force `RetrievalMode.GRAPH` to avoid spurious vector matches on long natural-language strings.

XI. END-TO-END FEATURE WALKTHROUGHS

1) **Friend recommendations.** The Explore view calls `api.graphFriendRecs(userId)`, which issues a typed client request to the read-only `GraphQueryService` path. The Chat view submits natural-language friend requests with `dataset` and `user_id` context; the pipeline reuses the same `GraphQueryService` logic before synthesis.
2) **Connection Explorer.** `connections/page.tsx` calls `api.graphConnectionPath(a,b)` for structured shortest-path, mutual-friend, and co-like rows, and may call `api.explainConnection` to obtain a pipeline-generated narrative. The graph call is Cypher-only; the explainer call invokes the multi-agent layer.
3) **Shortest path via chat.** Phrases such as "shortest path of friends between users with user id = 1 and user id = 2" are detected in `connection_path_nl.py`; two IDs are taken by regex or Gemini extraction; `get_connection_path` results are returned as `intent=connection_path` with a single structured row.
4) **Admin natural-language insert.** The admin flow calls the insert service with a `confirm` flag: preview mode returns a dry-run of parameterized Cypher; confirmed execution runs validated `MERGE` with dataset scoping.
5) **Dataset operations.** Operators can trigger re-ingest of SNAP-style bundles into Neo4j from the service layer; the health payload surfaces per-dataset node and edge counts for verification.

XII. ILLUSTRATIVE JSON EXAMPLES

Health and diagnostics JSON (abridged):

```
{ "status": "healthy", "neo4j_connected": true,
  "vector_backend": "neo4j",
  "gnn_datasets": ["facebook","twitter","reddit"],
  "dataset_counts": {
    "facebook": {"users": 22470, "edges": 171002}
  }
}
```

Conversational request body for a friend-recommendation question (abridged):

```
{ "message": "Friend recommendations for user_id = 1",
  "dataset": "facebook", "mode": "hybrid",
  "top_k": 10, "gnn_dataset": "facebook" }
```

The corresponding `ChatResponse` includes `results` with `id`, `name`, `mutual_friends`, `influence_score`, `follower_count`, and `insight` text.

Connection path JSON (abridged):

```
{ "shortest_path": {
    "node_names": ["A","B","C"],
    "rel_types": ["FRIEND","FRIEND"], "hops": 2 },
```

Table IV
ARCHITECTURAL DECISIONS AND RATIONALE.

Concern	Decision
Train/Inference	Separate: GPU training on Kaggle, CPU-only inference in API
Retrieval	Neo4j-native hybrid (FAISS removed)
Fusion	Reciprocal Rank Fusion (parameter-free)
Hallucination	6-step validator pipeline
LLM	Optional (degrades gracefully without API key)

```
  "common_friends": [...],
  "common_liked_posts": [...] }
```

XIII. ENVIRONMENT AND OPERATIONAL CONFIGURATION

Key variables (see project root `.env` and `api/bootstrap/config.py`): `NEO4J_URI`, `NEO4J_USER`, `NEO4J_PASSWORD`, `NEO4J_DATABASE`; `GEMINI_MODEL`, `USE_LLM` (disables LLM synthesis and NL extraction when set to `false`); `AUTO_INGEST`, `SEED_DEMO_NEO4J`; embedding model via `get_text_engine`; frontend `NEXT_PUBLIC_API_URL`. GNN weights live under `weights/` and are loaded by `inference_manager`.

XIV. DEPLOYMENT INFRASTRUCTURE

A. Docker

The system is containerized via `docker-compose.yml` with two services:

- **neo4j:** Neo4j 5.13 Community with APOC plugin, 512 MB–2 GB heap, 512 MB page cache, health check via `cypher-shell`.
- **api:** Python 3.11-slim, `uvicorn` server on port 8000, pre-trained weights mounted read-only, health check via `curl`.

B. Kaggle Integration

Training scripts are executed on Kaggle GPU (NVIDIA Tesla T4) notebooks. The `kaggle/` directory contains kernel metadata for automated push-and-run via `scripts/push_kaggle_kernel.py`. Outputs (`model_weights_*.pth`, `embeddings_*.npy`) are downloaded to the local `weights/` directory.

C. Key Design Decisions

XV. TESTING AND EVALUATION

A. Automated Tests

The `tests/` tree includes `test_all.py` (unit and integration: service-level smoke tests for the HTTP layer, `HybridRetriever` RRF fusion, query analyzer, synthesizer, validator) and `testing_pipeline_accuracy.py` (loads saved GNN weights, stratified split, optional OneCycle fine-tuning, minimum accuracy gate). From the repository root:

```
pytest tests/test_all.py -v
```

B. Reproducibility and Deployment

Docker Compose in `docker/docker-compose.yml` starts Neo4j 5.13 and the API service; Kaggle remains the source of record for GPU training; dataset ingest is idempotent with markers under each dataset directory.

XVI. CONCLUSION AND FUTURE WORK

We presented a GraphRAG-style social intelligence system integrating GNNs, Neo4j-backed hybrid retrieval, and LLM synthesis. Key findings:

- GNN+RAG consistently outperforms GNN-only: +1.3% on Facebook, +10.2% on Twitter.
- Semantic text features from SentenceTransformers provide discriminative power beyond structural graph features alone.
- The multi-agent KAG pipeline enables natural-language interaction with complex graph analytics.

Future work: Scaling to larger graphs (Reddit dataset scaffolded), latency optimization, GAT architectures for all datasets, real-time graph updates, and user-facing confidence calibration.

ACKNOWLEDGMENT

The authors thank Shiv Nadar University and the Department of Computer Science for resources and support for this course project.

REFERENCES

[1] "Graph Machine Learning in the Era of Large Language Models," *ACM Comput. Surv.*, 2025.
[2] "Facebook Graph Classification — Project Documentation," training log, GraphRAG Social Intelligence repository.
[3] "Neo4j-Only Architecture: Design Decisions & Performance Analysis," GraphRAG Social Intelligence repository, `ARCHITECTURE.md`.
[4] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NeurIPS*, 2017.
[5] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
[6] Neo4j, "Vector indexes," documentation, https://neo4j.com/docs/cypher-manual/current/indexes-for-vector-search/.
[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP*, 2019.
[8] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017.
[9] P. Veličković et al., "Graph attention networks," in *Proc. ICLR*, 2018.
[10] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.